

Codex Memory Systems Evaluation

<https://gemini.google.com/app/9cf40e526d920e70>

User prompt: give me an overview of memory systems for OpenAI Codex. make a special mention of any which implement a Karpathy style markdown wiki. give me the pros and cons, and make a recommendation, giving particular weight to minimising token consumption.

Response: I've put together a research plan. If you need to update it in any way, let me know.

User prompt: Start research

Response: I've completed your research. Feel free to ask me follow-up questions or request changes.

User prompt: you wrote this for GPT 5, but I've been using Astra in Codex. how does this assessment change in that case. also, please explain in detail the limits for Astra use in an OpenAI plus account.

Response: Switching your underlying model in OpenAI Codex from GPT-5.6 Sol to **GPT-6 Astra** changes several operational mechanics, particularly around context window limits, harness features, and token burn rates.

How the Memory Assessment Changes for GPT-6 Astra

1. Native "Notes and Search" vs. Traditional Compaction

Unlike previous models that relied on auto-compaction (summarizing or truncating earlier turns once context filled up), Astra introduces native persistent **Notes and Search** inside the Codex harness. When context fills, Astra writes notes across context windows and can dynamically search back into earlier conversation history and tool outputs.

- **Impact:** Astra reduces the risk of *immediate* context rot caused by naive transcript pruning.
- **Caveat:** Native notes are internal to the Codex session harness. They do not persist as human-readable codebase documentation or share state with other developer tools.

2. The 1.05 Million Context Window and Token Economics

GPT-6 Astra supports a maximum context window of **1,050,000 tokens** (with a 128,000-token output limit). However, expanding your active context window carries severe trade-offs:

- **The 272K Pricing Threshold Remains:** For token billing and API usage, requests exceeding 272,000 input tokens are still penalized with a 2x input rate and 1.5x output rate multiplier for the entire request.
- **Accelerated Token Burn:** Astra is a significantly heavier model that executes multi-step reasoning, tool interactions, and computer-use loops. Loading huge contexts into Astra increases prompt processing overhead per turn, causing plan allowances to drain exponentially faster.

3. Re-Evaluating Memory Paradigms Under Astra

- **Karpathy-Style LLM Wiki: Still Strongly Recommended.** While Astra's native notes help within a single long thread, a Karpathy-style Markdown Wiki (e.g., via `ai-memory`) provides three critical

advantages Astra cannot replicate natively:

1. **Multi-Agent Interoperability:** Plain-text Git Markdown files can be read and updated by Codex, Claude Code, and OpenCode simultaneously.
 2. **Human Auditing:** Every entry, Architectural Decision Record (ADR), and failure mode remains inspectable in Obsidian or VS Code.
 3. **Context Funneling:** It keeps active context payloads small by loading only pre-compiled summaries, protecting you from rapidly hitting account usage ceilings.
- **Vector Chunk-RAG:** Remains the least favorable option for individual coding agents due to code fragmentation and context noise.
 - **AST Code Mapping (Tree-sitter):** Remains an essential Tier 1 layer for low-cost, structural symbol awareness (~1k–2k tokens).

Detailed Limits for Astra Use on an OpenAI Plus Account

Using Astra within Codex on a standard **ChatGPT Plus account** introduces strict rate-limiting and quota behaviors:

1. Estimated Message Allowance

OpenAI officially estimates that a ChatGPT Plus account receives **5 to 45 local Astra messages per 5-hour rolling window**.

- This is **not** a guaranteed counter of 45 turns; it is an estimated planning range based on standard lightweight prompts.
- A secondary **weekly account cap** also applies.

2. Practical Quota Burn Rate in Codex

In practice, Codex sessions burn through the Plus allowance much faster than standard chat conversations due to tool execution and reasoning depth:

- **High Token Density:** Every turn where Codex reads multiple repository files, executes terminal commands, or runs test suites sends thousands of context tokens back to Astra.
- **Observed Depletion:** Real-world usage shows that **1 to 3 non-trivial Astra turns** (e.g., asking it to inspect a repo, write code, run tests, and fix errors) can consume **50% to 100% of a Plus user's entire 5-hour quota** in just a few minutes.
- **Shared Allowance:** Astra usage across ChatGPT, Work, and Codex draws from the exact same account allowance pool.

3. Default Working Context Window Cap

Although Astra natively supports 1.05M tokens, the default effective working context window inside Codex for Plus accounts remains capped at **272,000 tokens** (yielding ~258,400 usable input tokens after the default 95% safety multiplier).

- While you can manually override `model_context_window` in `config.toml` to expand the context, doing so on a Plus account will exhaust your 5-hour message allowance almost instantly due to the

massive increase in cached prompt tokens per turn.

Key Takeaway for Plus Users Running Astra

Because GPT-6 Astra on a Plus account has a tight usage quota, adopting the **Layered Context Funnel** (using an `AGENTS.md` file, an AST repo map, and a **Karpathy Markdown Wiki**) is even more critical than before. Keeping prompt sizes small ensures Astra turns consume as little quota allowance as possible, allowing you to complete actual engineering tasks before triggering the 5-hour lockout.

User prompt: would your recommendation work if I wanted to alternate between Astra and Gemini to work on the same project?

Response: **Yes, the recommendation works extremely well when alternating between Astra (OpenAI Codex) and Gemini.** In fact, cross-model interoperability is one of the strongest reasons to choose a Karpathy-style Markdown Wiki architecture over model-native internal memory.

While proprietary session features (like Astra's internal notes or Gemini's cloud-managed review memory) are locked inside their respective vendor runtimes, a Karpathy-style Markdown Wiki relies on plain-text files stored directly in your Git repository. Plain text serves as an open, vendor-agnostic memory layer.

Why the Architecture Works Across Both Models

1. Shared Project Memory

When Astra encounters a non-obvious bug or makes an architectural decision during a coding session, it writes that insight into a Markdown file inside `.memory/wiki/` (e.g., `wiki/gotchas.md` or `wiki/adrs/`). When you open your next session using Gemini CLI or Gemini Code Assist, Gemini reads `wiki/index.md` and inherits Astra's newly compiled knowledge immediately. Neither model suffers from "agent amnesia" when hand-offs occur.

2. Universal Protocol via MCP

Both OpenAI Codex and Gemini CLI natively support the Model Context Protocol (MCP). By running a shared local MCP server (like `ai-memory` or `memwiki`), both Astra and Gemini use the exact same tool primitives (`wiki_read`, `wiki_write`, `memory_ingest`, `memory_query`) to interact with your local wiki vault.

3. Symmetrical Token Efficiency

Although Gemini models feature massive context windows (often 1M to 2M tokens), feeding massive raw context into Gemini still incurs high latency and "lost in the middle" degradation. The Layered Context Funnel keeps prompt payloads small for **both** models, preserving Astra's 5-hour Plus quota while keeping Gemini's response times fast.

How to Set Up Dual Codex (Astra) & Gemini Compatibility

To alternate seamlessly between Astra and Gemini on the same codebase, implement the following brief configuration setup:

Step 1: Standardize Instruction Discovery

Codex looks for `AGENTS.md` at the repository root by default. Gemini CLI looks for `GEMINI.md` by default. You can unify them in one of two ways:

- **Option A (Native Mapping):** In Gemini CLI's configuration (`.gemini/settings.json`), set the agent to use `AGENTS.md` directly:

```
{
  "context": {
    "fileName": "AGENTS.md"
  }
}
```

- **Option B (File Inclusion):** Maintain a thin `GEMINI.md` file that imports or references `AGENTS.md`.

Step 2: Instruct Both Agents to Guard the Wiki

In your shared instruction file (`AGENTS.md` / `GEMINI.md`), add an explicit memory clause:

```
"Always search .memory/wiki/index.md before attempting complex refactoring. If you
discover an environment quirk, library bug, or make an architectural decision during this task,
record it in .memory/wiki/ before finishing. Do not rely on session-private memory."
```

Step 3: Use Gemini CLI's Memory Management Utilities

When working in Gemini CLI, you can inspect and force refreshes of the compiled Markdown context using built-in slash commands:

- `/memory show`: Displays the exact concatenated instructions and wiki summaries currently loaded into Gemini's context window.
- `/memory reload`: Forces Gemini to re-read updated `.memory/wiki/` files mid-session if Astra or a background process updated them.

Step 4: Track Memory via Git

Because memory updates exist as Markdown diffs, commit changes in `.memory/wiki/` to Git alongside your code changes. This ensures that teammates—or automated CI/CD agents using either Gemini or Astra—always run on the exact same institutional knowledge base.

User prompt: ok, please give me a precise implementation plan

Response: Here is a step-by-step implementation plan to establish a dual-compatible, token-efficient memory architecture for alternating between **OpenAI Codex (Astra)** and **Gemini CLI** on the same project repository.

Step 1: Initialize the Memory Repository Structure

Run the following commands in your project root to set up the three-layer Karpathy-style vault structure:

```
# Create directory hierarchy
mkdir -p .memory/raw .memory/wiki/adrs
```

```
# Create core index and ledger files
touch .memory/wiki/index.md
touch .memory/wiki/log.md
touch .memory/wiki/gotchas.md
touch .memory/wiki/failed_approaches.md
```

Populating Core Seed Files

1. `.memory/wiki/index.md` (The Map of Content):

Project Memory Index

System Architecture & ADRs

- [Architectural Decisions](#)

Execution Knowledge

- [Environment & Library Gotchas](#)
- [Failed Implementation Paths](#)

2. `.memory/wiki/log.md` (The Audit Trail):

Memory Update Log

[YYYY-MM-DD] INIT | Created project memory vault

Step 2: Establish Single-Source-of-Truth Instructions

To avoid maintaining two sets of project guidelines, use `AGENTS.md` as your primary instructional source and bridge Gemini CLI to it.

1. Create `AGENTS.md` at Repository Root

:

Repository Instructions & Memory Protocol

Context & Build Commands

- Build: `pnpm build`
- Test: `pnpm test`
- Lint: `pnpm lint`

Memory Management Protocol

1. BEFORE REFACTORING OR WRITING CODE:

- Read `.memory/wiki/index.md` and check `.memory/wiki/gotchas.md`.
- Search for prior failures in `.memory/wiki/failed_approaches.md` before re-attempting complex fixes.

2. DURING AND AFTER WORK:

- If you encounter a library bug, environment quirk, or make an architectural decision, immediately record it under `.memory/wiki/`.
- Log significant changes in `.memory/wiki/log.md` using the format: `## [YYYY-MM-DD] ACTION | Summary`.
- Never store architectural decisions or quirks solely in session-private memory.

2. Configure Gemini CLI to Ingest AGENTS.md Automatically

:

Create or update `.gemini/settings.json` in your project root:

```
{
  "context": {
    "fileName": "AGENTS.md"
  }
}
```

(Alternatively, create a root `GEMINI.md` with the single line: `@file AGENTS.md`).

Step 3: Configure the Shared MCP Memory Server

Both Codex and Gemini CLI need access to the same local Model Context Protocol (MCP) server (such as `ai-memory` or `memwiki`) to manage reads, writes, and linting.

1. Configure OpenAI Codex MCP

In `~/codex/mcp.json` (or your active Codex configuration):

```
{
  "mcpServers": {
    "memory": {
      "command": "npx",
      "args": ["-y", "ai-memory-mcp"],
      "env": {
        "MEMORY_PATH": ".memory"
      }
    }
  }
}
```

```
}
```

2. Configure Gemini CLI MCP

Register the exact same server inside Gemini CLI:

```
gemini mcp add memory -- npx -y ai-memory-mcp
```

(Verify connection in Gemini CLI using `gemini mcp list`).

Step 4: Configure Git Version Control & Safeguards

The compiled Markdown wiki must be shared between developer tools and teammates via Git, while avoiding tracking temporary local build states.

Add the following to your root `.gitignore`:

```
# Keep compiled knowledge tracked in Git
!.memory/raw/
!.memory/wiki/

# Ignore local temporary agent/SQLite indexes if generated by MCP runtime
.memory/*.db
.memory/tmp/
```

Step 5: Session Hand-Off Operational Protocol

Follow this simple workflow to alternate seamlessly between Astra and Gemini without context loss:

[Start Task in Codex (Astra)]

- Astra reads AGENTS.md & .memory/wiki/index.md [cite: 3, 5]
- Astra executes work & writes discoveries to .memory/wiki/ [cite: 1]
- Run ``git commit -m "docs(memory): update wiki via Astra"`` [cite: 2]

[Switch to Gemini CLI]

- Open Gemini CLI in the same repo
- Run ``/memory reload`` to ensure fresh disk reads [cite: 10]
- Gemini inherits all recorded GOTCHAs & ADRs [cite: 1]
- Gemini executes work & updates .memory/wiki/ [cite: 1]
- Run ``git commit -m "docs(memory): update wiki via Gemini"`` [cite: 2]

In-Session Checklist

- **When using Astra (Codex):**
 - Keep active prompts small by relying on Astra to read targeted `.memory/wiki/` files rather than dumping entire source files, preserving your 5-hour Plus quota.
 - Before closing the CLI, ensure Astra executes its `memory Consolidate` or `Finalize`

session tool pass.

- **When using Gemini CLI:**

- Run `/memory show` at session start to verify that `AGENTS.md` and the wiki index loaded cleanly.
- Run `/memory reload` if Astra updated any memory files in the background while Gemini was open.

User prompt: compare ai-memory to memwiki and make a concrete recommendation please. also, are both of them open source?

Response: **Yes, both ai-memory and memwiki are open-source projects.**

- **ai-memory** (by Fabio Akita) is licensed under the permissive **MIT License**. It is written in Rust and hosted on GitHub ([akitaonrails/ai-memory](#)).
- **memwiki** (by Swapnil) is also an open-source project hosted on GitHub ([hereisSwapnil/memwiki](#)) and distributed as an npx CLI package.

Comparison: ai-memory vs. memwiki

While both implementations build upon Andrej Karpathy's Markdown Wiki concept, they operate at fundamentally different architectural levels:

Feature / Dimension	memwiki	ai-memory
Architectural Nature	Protocol & Folder Scaffolder (Node/npx). It generates starter files and prompt instructions for your AI agent.	Active Memory Infrastructure & MCP Server (Rust binary). It runs as a local background engine and tool server.
Session Event Capture	Passive / Model-Dependent: Relies entirely on the LLM choosing to edit files before a session closes.	Active / Hook-Driven: Uses native lifecycle hooks (PostToolUse, SessionStop, PostCompaction) to capture tool outputs and errors automatically in the background.
Search Mechanics	Top-Down File Reading: The LLM reads <code>.memory/wiki/index.md</code> or relies on basic shell commands (grep).	Hierarchical + Hybrid Index: The agent reads Markdown files directly, but the server maintains a local, background-derived SQLite/full-text search index as a backstop.
Cross-Agent Handoffs	File-Based: Relies on both agents reading the same Git-tracked Markdown files.	Typed Handoff Protocol: Provides explicit, programmatic handoff mechanisms across 20+ agent CLIs (Codex, Gemini CLI, Claude Code, OpenCode).
System Overhead	Zero Runtime: No background daemons or containers required.	Lightweight Runtime: Runs as a single local binary or Docker container.

Concrete Recommendation

Choose **ai-memory** if you are alternating between **Astra (Codex)** and **Gemini CLI**.

Why **ai-memory** is the stronger choice for your setup:

1. **Quota Protection Against Astra Lockouts:** Because GPT-6 Astra on a Plus account can hit its 5-hour quota after just 1 to 2 turns, relying on Astra to manually remember and execute Markdown wiki updates before its turn finishes is risky. If Astra gets cut off mid-task due to a rate limit, **memwiki** will lose that turn's context. **ai-memory**'s background lifecycle hooks capture tool executions and failures automatically before sandbox cutoff.
2. **Explicit Cross-Vendor Handoff Protocol:** **ai-memory** was designed specifically to facilitate state handoffs between different agent harnesses. When you stop an Astra session mid-refactor and launch Gemini CLI, **ai-memory** exposes a structured `memory_handoff_accept` tool primitive so Gemini picks up the exact execution boundary.
3. **Hybrid Search Safeguard:** If your repository wiki expands beyond a few dozen pages, **memwiki** forces the LLM to manually scan directory structures. **ai-memory** provides a local full-text search backstop without requiring external vector database infrastructure.

When to consider **memwiki** instead:

Use **memwiki** if you want a **1-minute, zero-dependency setup**. If you do not want to run a local Rust binary or Docker container and simply want to run `npm run memwiki init` to drop clean prompt rules into your repository, **memwiki** is an excellent, lightweight starting point.

User prompt: two things... the first is, that the Karpathy wiki seems very centred on adding raw sources of knowledge to be processed. while I am not opposed to that, and will undoubtedly do it every now and then, I don't tend to do it often. I was more interested in the models keeping a memory of our conversations, design decisions, et cetera. also, if using a Karpathy style wiki, where would an article created by Gemini deep research go, or some markdown document created by a model for some purpose?

Response: 1. Memory for Conversations & Decisions (No External Sources Needed)

In personal research workflows, Karpathy-style wikis often start by ingesting external PDFs or API documentation. However, in **software engineering workflows**, your conversations, trial-and-error attempts, and debugging turns *are* the raw sources.

You do not need to manually feed external documents into the system. Instead, the wiki serves as an **accumulated decision log** that preserves context across session cutoffs:

- **Architectural Decision Records (`.memory/wiki/adrs/`):** When you and Astra discuss an architectural trade-off (e.g., choosing SQLite over PostgreSQL, or choosing a specific state management pattern), the agent writes a concise summary in `adrs/0002-state-management.md`. The next time you open a session with Gemini, Gemini reads that file and immediately understands *why* the codebase is structured that way without needing to re-read your old chat logs.
- **Execution Gotchas (`.memory/wiki/gotchas.md`):** When you spend 20 minutes debugging a weird SDK quirk or build flag, the agent appends a two-sentence note to `gotchas.md`.

- **Failed Paths (.memory/wiki/failed_approaches.md):** If you try a specific refactoring approach and run into a blocking limitation, recording that failure prevents either model from attempting the exact same invalid fix in a future turn.
- **Session Handoffs:** Tools like ai-memory use background hooks to record tool execution summaries and handoff states automatically, so you don't even have to manually ask the model to "remember" anything.

2. Where to Put Gemini Deep Research Reports & Model-Generated Markdown

Where an AI-generated document belongs depends on its role in your project:

Option A: Put it in .memory/raw/ (If it's an unedited reference report)

If Gemini Deep Research generates a multi-page market analysis, API specification, or technical comparison report that you want to keep as an unmodified source of truth:

1. Save the file to `.memory/raw/2026-09-stripe-v3-migration-research.md`.
2. Tell your agent: *"Read .memory/raw/2026-09-stripe-v3-migration-research.md, extract the core implementation steps, and update .memory/wiki/."*
3. The agent will summarize the actionable decisions into the wiki, leaving the raw report intact for reference.

Option B: Put it in .memory/wiki/ (If it's a domain/feature guide)

If the document is a distilled technical guide or module specification created specifically for your codebase (e.g., an Authentication System Overview):

1. Save it directly as a domain page, such as `.memory/wiki/auth-system.md`.
2. Add a link to it inside `.memory/wiki/index.md` so the agents can find it when navigating the repository.

Option C: Put it in standard project /docs/ (If it's permanent project documentation)

If Gemini creates a user-facing README, PRD (Product Requirement Document), or developer setup guide intended for human teammates:

1. Place it in your standard repository documentation folder (e.g., `/docs/architecture-overview.md`).
 2. Reference or link to it in `AGENTS.md` so the agents know it exists alongside the rest of your source code.
-